Web-Based Ground Control Station for Real-Time Monitoring Quadcopter UAVs

Noper Ardi[1], Said M Razali[2] and Ahmad Hamim Thohari[3]

Informatic Engineering, Politeknik Negeri Batam, Batam, Indonesia
noperardi@polibatam.ac.id

Abstract. This study presents the design, implementation, and evaluation of a novel web-based Ground Control Station (GCS) application for real-time monitoring of quadcopter Unmanned Aerial Vehicles (UAVs). The research addresses the operational limitations of traditional desktop-based GCS applications, such as platform dependency, installation overhead, and specific hardware driver incompatibilities, which hindered a previous C# application from reading flight controller communication ports and accurately visualizing UAV positions. To overcome these challenges, this work introduces a cross-platform, browser-accessible GCS developed using a full-stack JavaScript architecture, featuring Node.js for the backend and React.js for the frontend. Real-time, low-latency data streaming is achieved through a persistent WebSocket connection, managed by Socket.IO, which processes telemetry data from the flight controller via the MultiWii Serial Protocol (MSP). The system was developed using the Prototype method to ensure continuous alignment with user needs. Performance evaluation under controlled indoor conditions demonstrated an average end-to-end data transmission latency of 48.5 ms, a stable data refresh rate of 19.8 Hz, and a mean positional accuracy of ±1.2 m. The GCS application effectively displays critical sensor data-including pitch, roll, yaw, battery status, and UAV position on a digital map accurately and in real-time. This application offers a flexible, accessible, and efficient GCS alternative for UAV developers and is expected to support the Barelang Force 2.0 Team in the Indonesian Flying Robot Contest (KRTI).

Keywords: Ground Control Station; Quadcopter; Unmanned Aerial Vehicle; Real-Time Monitoring; Prototype Development.

1 Introduction

The advancement of Unmanned Aerial Vehicle (UAV) technology has accelerated significantly, with the quadcopter emerging as one of the most versatile and widely deployed platforms for applications ranging from aerial mapping to search-and-rescue missions. Central to the safe and effective operation of any UAV is the Ground Control Station (GCS), which serves as the command hub for monitoring the vehicle's flight status and receiving real-time telemetry data. An effective GCS is paramount for ensuring mission success and enhancing operational safety. This research is directly motivated by the challenges faced within the framework of the Indonesian Flying Robot

© The Author(s) 2025
H. Widiastuti et al. (eds.), *Proceedings of the 8th International Conference on Applied Engineering (ICAE 2025)*,
Advances in Engineering Research 290,
https://doi.org/10.2991/978-94-6463-982-7_9

Contest (KRTI), a key initiative fostering innovation in national UAV technology. Specifically, the Barelang Force 2.0 team from Politeknik Negeri Batam encountered critical operational failures with their prior GCS, a desktop application developed in C#. The primary issues included an inability to reliably read the flight controller's communication port and a malfunctioning map visualization, which are symptomatic of the architectural limitations inherent in many traditional desktop GCS applications that suffer from deep dependencies on host operating system drivers and APIs[1].

This context necessitates a broader examination of the current GCS landscape, which is dominated by powerful open-source solutions such as Mission Planner and QGroundControl. While these platforms are industry standards, they present their own limitations. Mission Planner is a Windows-only application, restricting its utility in heterogeneous development environments, while QGroundControl, despite its excellent cross-platform support, requires local installation and configuration like any desktop-native application. Both are predominantly built around the MAVLink protocol, which may be overly complex for projects utilizing flight controllers that support simpler protocols like the MultiWii Serial Protocol (MSP)[2][3]. A review of current literature reveals a distinct research gap: a lack of lightweight, truly platform-agnostic GCS solutions that operate entirely within a standard web browser, require no installation, and are built on modern web stacks. While web platforms for post-flight data analysis exist, they are not designed for real-time, direct-to-UAV monitoring.

A study in [3] indicates that modern GCS requires three main features: Mission Control System (MCS), Payload Control System (PCS), and Communication System (CS). Such systems are generally developed as desktop applications using Microsoft Visual Studio with C#. However, this approach has limitations in terms of platform flexibility and accessibility. Similarly, [4] presented the development of a GCS for multi-UAV systems supporting multi-GCS connections, allowing multiple operators to work within a single mission. Their GCS was developed in C++ using libraries such as SDL, OpenGL, BOOST, and STL. While the system provided comprehensive functionalities, its development and maintenance complexity posed significant challenges.

A portable GCS telemetry system for controlling and monitoring the GALAK-24 UAV was proposed in [5[[6], demonstrating its effectiveness in real-time data transmission, flight status monitoring, and image capture. Although capable of executing precise mission commands, the system was still desktop-based, limiting mobility and accessibility. Unlike previous studies that primarily employed desktop applications using C# or C++, this research proposes the development of a web-based GCS application using full-stack JavaScript technologies. This approach offers several advantages over conventional desktop applications. First, web-based applications provide higher accessibility as they can be accessed across devices without specialized installation, provided the device has a web browser. Second, development with a full-stack JavaScript environment (Node.js for backend, React.js for frontend, and MongoDB for database) enables faster and more efficient development, leveraging the npm ecosystem rich with libraries

and frameworks. Third, JavaScript's asynchronous capabilities are well-suited for handling real-time communication and intensive data processing, which are critical in UAV monitoring [2].

This paper presents a novel GCS that directly addresses the identified research gap by leveraging the accessibility and flexibility of web technologies. The primary contributions of this work are the development of a fully web-based GCS architecture using a full-stack JavaScript environment (Node.js and React.js), the implementation of a low-latency, real-time data pipeline using the WebSocket protocol, and the successful integration of the MultiWii Serial Protocol (MSP) within this architecture. This approach offers inherent advantages in cross-platform accessibility and zero-installation deployment. The scope of this research is focused on the development and evaluation of a real-time monitoring system; functionalities are limited to displaying telemetry and sensor data. Advanced features such as flight control, waypoint navigation, and mission planning are identified as areas for future work.

2 Methodology

2.1 System Hardware configuration

The experimental setup for this research comprises the UAV itself and the GCS host computer. The specifications for each component were chosen to be representative of a typical setup for a competitive or research-oriented quadcopter project. The detailed hardware specifications are provided in Table 1. This level of detail is crucial for ensuring the reproducibility of the experimental results and providing context for the system's performance evaluation. The flight controller, an F7 Mini, was specifically selected for its robust support of the MultiWii Serial Protocol (MSP), which is central to the data acquisition process in this study [7]. The GCS host computer is a standard developer-grade laptop, demonstrating that the system does not require specialized or high-end hardware to operate effectively.

Table 1. System Hardware Specifications

Component Category	Item	Specification
UAV Platform	Quadcopter Frame	Q250 Frame, 250 mm wheelbase
	Flight Controller	F7 Mini Flight Controller
	Motors	4x Brushless 2205 2300KV
	ESCs	60A 4-in-1 BLHeli_S
	Battery	4S 1500mAh LiPo (14.8V)
	Onboard Sensors	Integrated IMU (Gyroscope, Accelerometer), Barometer, Magnetometer, GPS Module
GCS Platform	Host Computer	Laptop, Intel Core i7-10750H, 16GB RAM
	Operating System	Ubuntu 22.04 LTS
	Communication Link	USB 2.0 Type-A to Micro-USB Cable

2.2 System Overview

The Ground Control Station (GCS) application is designed to monitor data received from the quadcopter to ensure safe and efficient operations. The application usage begins with the operator logging in to access the features of the GCS system. After logging in, the operator is directed to the System Monitor page. At this stage, the operator must connect the quadcopter to the GCS system via a USB cable. On the System Monitor page, the system displays sensor data acquired from the quadcopter, such as pitch, roll, yaw, and battery status. In addition, the GCS application provides a Map View feature, enabling the operator to monitor the quadcopter's position in real-time through a map[8]. This feature presents the quadcopter's location relative to the GPS coordinates and displays additional information obtained from the GPS sensor. To support more comprehensive monitoring, the GCS system also presents Inertial Measurement Unit (IMU) data, which includes gyroscope and accelerometer readings. All features within this application are designed to ensure that the quadcopter operates optimally while providing the operator with precise control. An overview of the system is illustrated in Figure 1.

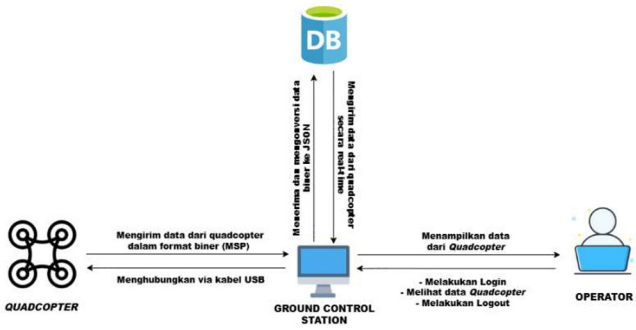


Fig.1. System Overview

2.2. Software Architecture and Data Flow

The software architecture is designed as a modern, three-tier web application. It consists of the data source (UAV Flight Controller), a backend server (Node.js), and a frontend client (React.js). This architecture was chosen to decouple the hardware interface from the user interface, promoting modularity and scalability.

2.2.1. Data Acquisition and Communication Protocol (Backend)

The foundation of the data pipeline is the communication between the flight controller and the backend server. The flight controller transmits telemetry data using the Multi-Wii Serial Protocol (MSP), a lightweight, binary message-based protocol designed for efficient communication over serial links. MSP is supported by a variety of flight controller firmwares, including Betaflight, iNav, and ArduPilot (for OSD and telemetry functions), making it a versatile choice.

The backend is implemented in Node.js, a JavaScript runtime environment chosen for its asynchronous, non-blocking I/O model, which is highly suitable for handling real-time data streams from hardware devices. The serialport library is utilized to establish a connection with the flight controller's virtual COM port via USB at a baud rate of 115200[9].

Upon connection, the backend begins listening for incoming data. It implements a parser that processes the raw binary stream from the serial port. The parser identifies the MSP message structure, which consists of a preamble ('$M<'), a direction indicator, payload size, message ID, the payload itself, and a checksum. The backend decodes the binary payload based on the message ID (e.g., MSP_ATTITUDE, MSP_RAW_GPS) into meaningful numerical values. These parsed values are then structured into a standardized JSON (JavaScript Object Notation) format, making the data easily consumable by any web client [10].

2.2.2. Real-Time Data Streaming (Backend to Frontend)

To achieve real-time data visualization, a persistent, low-latency communication channel between the backend and frontend is essential. Traditional HTTP-based REST APIs, which operate on a request-response model, are ill-suited for this purpose. An HTTP polling approach, where the client repeatedly requests updates from the server, would introduce significant latency and network overhead from the continuous setup and teardown of TCP connections and redundant HTTP headers for each request.

This system instead employs the WebSocket protocol, which provides a full-duplex communication channel over a single, long-lived TCP connection. This allows the server to "push" data to the client the instant it becomes available, drastically reducing latency and overhead. The implementation uses the Socket.IO library, which provides a robust abstraction over WebSockets with features like automatic reconnection and event-based communication. Once the Node.js backend parses a new MSP message and converts it to JSON, it immediately emits this data object over the WebSocket channel to all connected frontend clients [11].

2.2.3. Data Visualization (Frontend)

The frontend client is a Single-Page Application (SPA) built with React.js, a popular JavaScript library for building dynamic and interactive user interfaces. Its component-based architecture allows for the modular development of UI elements, each responsible for displaying a specific piece of telemetry data.

The client uses the socket.io-client library to establish a connection to the backend server and listen for incoming data events. When a new telemetry packet is received, the application's state is updated, and React efficiently re-renders only the components that have changed [12]. This ensures a smooth and responsive user experience. Key visualization components include:

1) Sensor Status Indicators: Displaying the health of the gyroscope, accelerometer, magnetometer, barometer, and GPS.
2) Attitude and Heading Indicators: Providing a visual representation of the UAV's orientation.
3) Data Charts: Real-time graphs plotting the pitch, roll, and yaw angles over time.
4) Battery Information: Displaying voltage, current, capacity used, and percentage remaining.
5) Map View: Using the Leaflet.js library, an open-source interactive map library, to plot the UAV's real-time latitude and longitude coordinates on a map tile layer.

2.3 Software Development Method

The development of this GCS application followed the Prototype Model. This software development lifecycle model involves creating an initial, functional version of the system (a prototype), which is then evaluated by users to gather feedback for subsequent refinement and iteration.

While other methodologies like Waterfall and Agile are common, the Prototype model was specifically chosen for several reasons pertinent to this project's context. The Waterfall model, with its rigid, sequential phases and requirement for a complete, upfront definition of all system specifications, is unsuitable for research and development projects where requirements are often discovered and refined through experimentation. The primary stakeholder for this GCS is a competitive robotics team whose needs evolve based on competition rules, field testing, and iterative hardware changes [13].

Agile methodologies, while iterative, are typically structured around sprints aimed at delivering discrete, pre-defined features from a product backlog. The Prototype model, in contrast, is better suited for the initial, exploratory phase of this project. The primary goal was not to deliver a set of features, but to quickly build a working system to validate the core technical concept the viability of a web-based GCS and to use this tangible prototype as the primary tool for eliciting detailed requirements and usability feedback from the end-users (the Barelang Force 2.0 team). This iterative feedback loop, centred on a working model, ensures the final product is closely aligned with user needs and is more effective for requirement discovery in an R&D setting than the more structured approaches of Waterfall or Agile.

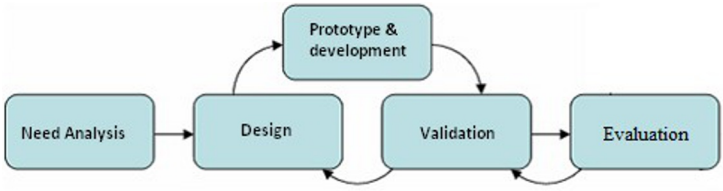


Fig. 2 Rapid Prototype Method

The Prototype Method is a software development approach that involves creating an initial prototype of the system. This prototype is used to illustrate the preliminary design, explore potential issues, and obtain user feedback to refine the design further. This approach enables intensive collaboration between developers and users, thereby ensuring that the developed system aligns more closely with user needs. The method has proven effective in minimizing errors and improving the quality of the final product through continuous iterations, ranging from mock-up creation to system coding in accordance with user evaluations [7].

In this study, the prototype method is applied to produce an initial representation of the application to be developed. The process begins with designing the application wireframe, which is then evaluated by users. The results of this evaluation serve as a reference for software developers in building the final application.

2.4 System Design

Software system design requires a structured and systematic approach to ensure that the final outcome meets the desired requirements. One of the effective methods for documenting and visualizing system design is the Unified Modeling Language (UML) [7][14]. UML provides a set of diagrams that allow designers to visually represent various system elements, including static structures, dynamic behaviors, and interactions between system components. Diagrams used in UML, such as use case diagrams, sequence diagrams, and activity diagrams, are highly useful in defining system functionalities, managing workflows, and illustrating interactions between users and the system. By utilizing UML, the software design process becomes more organized, facilitates communication among team members, and enables more structured design documentation. The application of this method plays a crucial role in producing efficient and effective systems.

2.4.1 Functional & Non-Functional Need

Table 2. Functional Need.

Kode	Keterangan
FR-01	Operator can create account
FR-02	Operator can login
FR-03	Operator can choose port
FR-04	Operator can see quadcopter sensor status
FR-05	Operator can see quadcopter battery status
FR-06	Operator can see quadcopter pitch orientation angle
FR-07	Operator can see orientation angle of the quacopter.
FR-08	Operator can see quadcopter yaw orientation angle
FR-09	Operator can see quadcopter position on the map
FR-10	Operator can see flight indicator attitude
FR-11	Operator can see flight indicator heading
FR-12	Operator can see GPS quadcopter status

This study also takes into account non-functional aspects to ensure the system operates optimally, as shown in Table 2 below.

Table 3. Non-Functional Need.

Criteria	Parameter
Availability	The system must be able to handle network or server failures quickly, so that the operator can continue monitoring the quadcopter without significant disruption.
Reliability	The system must be able to handle real-time data from the quadcopter, especially when there is rapid movement or changing conditions.
Portability	The system should be able to run smoothly on various browsers without any compatibility issues.
Interoperability	The system must be configurable and integrated with various types of UAVs other than quadcopters.
Language	Using English.

2.4.2 Sequence Diagram

The Sequence Diagram is one of the diagrams in the Unified Modeling Language (UML) used to illustrate interactions between objects within a system. This diagram depicts the chronological order and messages exchanged between objects during the execution of a scenario. Sequence diagrams are highly useful for demonstrating how objects in the system interact in detail at each step of the process, enabling developers to design precise communication flows and timing among objects in an application or software under development. In addition to showing the order of interactions, Sequence Diagrams also serve to identify potential issues in object-to-object communication, thereby allowing the system to be designed more efficiently [10][15].

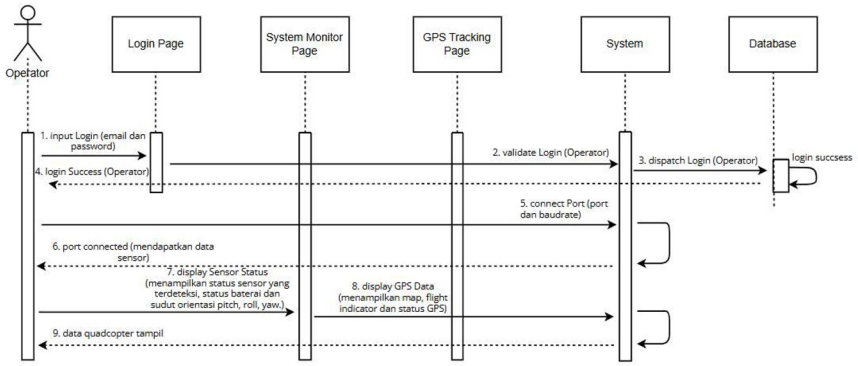


Fig.3 Sequence Diagram

The Sequence Diagram in Figure 3 above illustrates the communication flow among various system components, starting from the operator login to sensor data retrieval and the display of GPS information. The process begins with the operator entering an email and password to log into the system. Once the login is successful, the system displays the System Monitor page to observe sensor status, battery status, and orientation data

such as pitch, roll, and yaw. The operator then selects and connects the appropriate port to acquire sensor data. After the port is connected, the system begins to display battery status data and orientation angles (pitch, roll, and yaw). Subsequently, the system presents location information on the map, flight indicators, and GPS status, which can be utilized by the operator to monitor flight conditions. This sequence ensures a structured workflow for real-time monitoring and effective control of the quadcopter.

3 Result and Discussion

3.1 System Implementation

The Login page shown in Figure 4 presents the login interface of the Ground Control Station system. The interface is designed with a minimalist and professional layout to facilitate the user authentication process. Users are required to enter their email address and password in the two input fields provided. Below these fields, a *Sign in* button is available to process the login. In addition, a *Create account* link is provided for new users who have not yet registered.

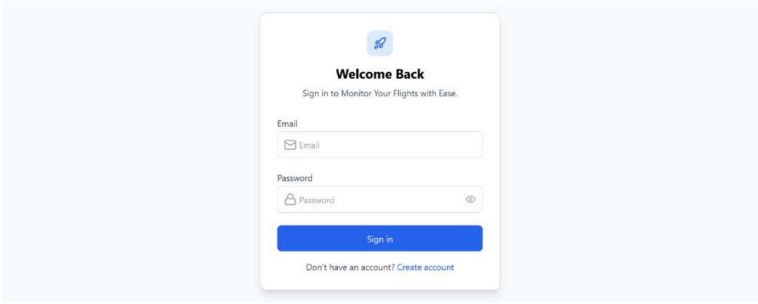


Fig. 4. Landing Page

The registration page shown in Figure 5 illustrates the user registration interface of the Ground Control Station system. Users are required to complete three input fields: Full Name, Email, and Password. Each input field is accompanied by an icon to enhance clarity and usability. Once the data is entered, users can press the *Create Account* button to register a new account. At the bottom, an alternative link is provided for users who already have an account, allowing them to directly access the login page by selecting *Sign in*.

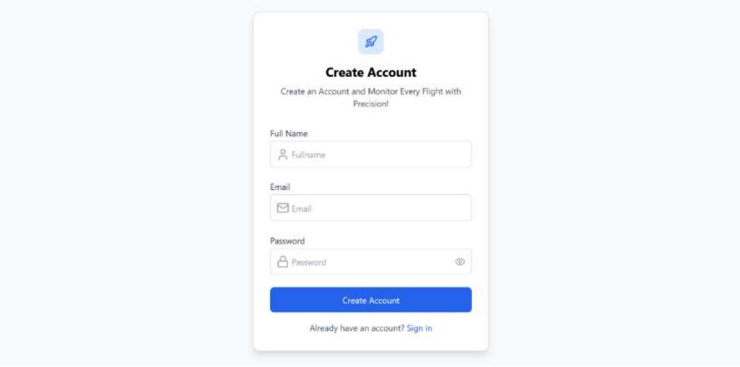


Fig. 5. Registration Page

The System Monitor page shown in Figure 6 presents the system interface used to monitor the operational status of the quadcopter. In the header section, there is a communication port configuration that connects the device with the system for data transmission. The baud rate is set to 115200, ensuring optimal data transfer speed. The Connect button is used to initiate the connection between the quadcopter and the system, enabling real-time transmission of sensor data and quadcopter status. This page provides essential information from various sensors supporting quadcopter operations, including the gyroscope, accelerometer, magnetometer, barometer, and GPS. Each sensor is equipped with color-coded indicators to reflect its status: functioning sensors are marked in green, while failed sensors are marked in red. Additionally, the page displays quadcopter status information such as levelled position, safe navigation, and hardware health, also indicated in green or red [16].

The gyroscope is a sensor designed to measure changes in position or object rotation in three-dimensional space. This technology is used to ensure accurate orientation in UAVs, avoid electromagnetic interference that could affect other sensor performance, and maintain stability during operation—particularly in applications requiring high precision, such as navigation systems and augmented reality [17]. The accelerometer is used to measure linear acceleration, and in the context of UAVs, it helps determine angular position by detecting acceleration due to gravity. However, its main challenge is measurement noise, which can affect accuracy. To address this, accelerometer data is often processed using digital filters such as the Kalman filter or complementary filter to enhance angular estimation accuracy [12]. The magnetometer, as a magnetic field sensor, improves navigation accuracy by detecting the Earth's magnetic field, which is critical for UAV orientation and positioning. Magnetometers are often integrated with the Inertial Measurement Unit (IMU) to provide more accurate data, especially when GPS signals are disrupted [18]. The barometer is employed to measure atmospheric pressure and is crucial in UAV flight systems. By measuring air pressure, the barometer can determine UAV altitude and help maintain flight stability even in environmental conditions such as wind. Barometric sensors like the BME280 are commonly used to ensure UAVs follow programmed flight paths accurately [14].

This page also displays data related to battery status, including voltage, battery level, consumed capacity, electrical current, and graphical representations of the quadcopter's pitch, roll, and yaw values. Pitch refers to the tilt of the rotor, which affects flight performance and stability. Proper pitch adjustment allows better maneuverability and higher energy efficiency, which is vital for monitoring and surveillance applications requiring operational accuracy [15]. Roll is the UAV's rotation around its longitudinal axis, enabling left or right movement. Efficient roll adjustment is crucial to maintaining stability and responsiveness, particularly in windy conditions or during high-speed maneuvers. Gyroscope sensors detect and adjust roll angles in real time, enhancing navigational accuracy and preventing loss of control [15]. Yaw refers to the rotation of the quadcopter around its vertical axis, allowing it to turn left or right. Proper yaw adjustment is essential for maintaining stability and maneuvering accuracy, especially in confined spaces. Orientation sensors such as gyroscopes and magnetometers are used to detect and control yaw angles in real time, ensuring flight balance and precision [20].

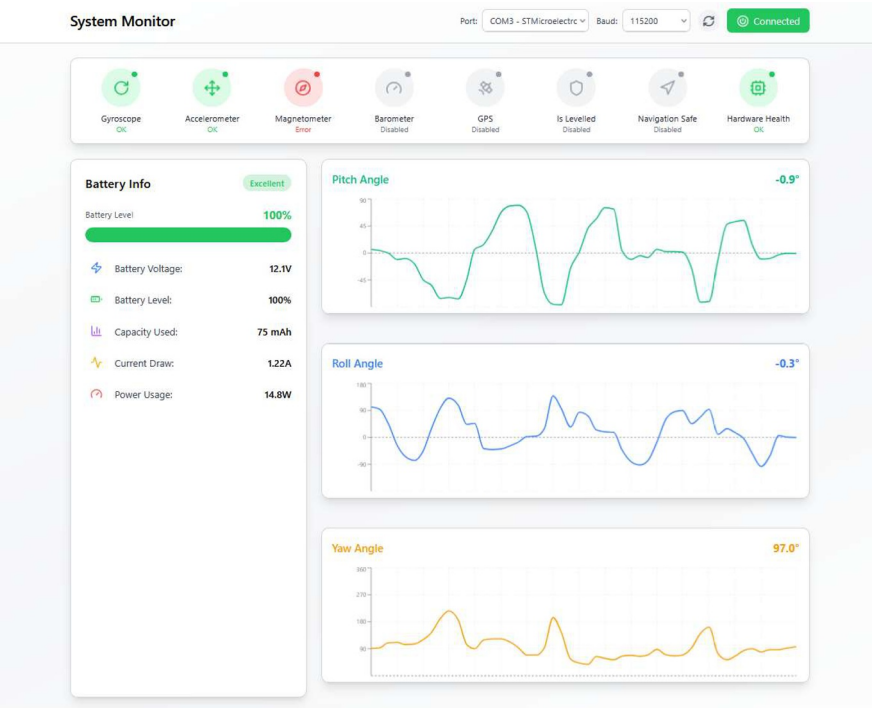


Fig.6. system Monitor Page

The GPS Tracking page shown in Figures 7 and 8 displays the real-time position of the quadcopter within an interactive map. In the header section, there are communication port settings that connect the quadcopter with the system for data transmission. The baud rate is set to 115200, ensuring optimal data transfer speed. The Connect button is used to initiate the connection between the device and the system, enabling real-time

transmission of GPS data and quadcopter information. On this page, the quadcopter's current location is displayed on a detailed map that includes information about the surrounding area, such as street names and other significant landmarks. Additionally, this page provides a flight attitude indicator that shows the tilt and orientation of the quadcopter, along with a heading indicator that displays the direction the quadcopter is facing. Furthermore, this page presents GPS-related information such as altitude, which describes the quadcopter's flight height. Altitude plays a crucial role in quadcopter route planning, particularly for large-scale mapping missions. Maintaining consistent altitude allows the quadcopter to collect more accurate data, especially when operating collaboratively with other UAVs [21].

Latitude is also displayed as a geospatial parameter used to determine the geographical position in quadcopter missions. The use of latitude, together with longitude, enables path-planning algorithms to design efficient routes, optimize mission time, and prevent UAV collisions, particularly during operations over wide areas [21]. Longitude represents another key component for determining geographical location in quadcopter route planning. In this study, longitude is used in combination with latitude to define destination points within the global coordinate system. Longitude data allows calculation of distances between route points that the quadcopter must traverse. By transforming coordinates into relative distances, the quadcopter can follow an optimal path to reach the designated area. The use of longitude in path-planning algorithms is also essential to avoid UAV collisions, thereby enhancing mission efficiency and safety [21]. In addition, other information such as speed, the number of connected satellites, and distance to the home point (dist to home) is also presented on this page.

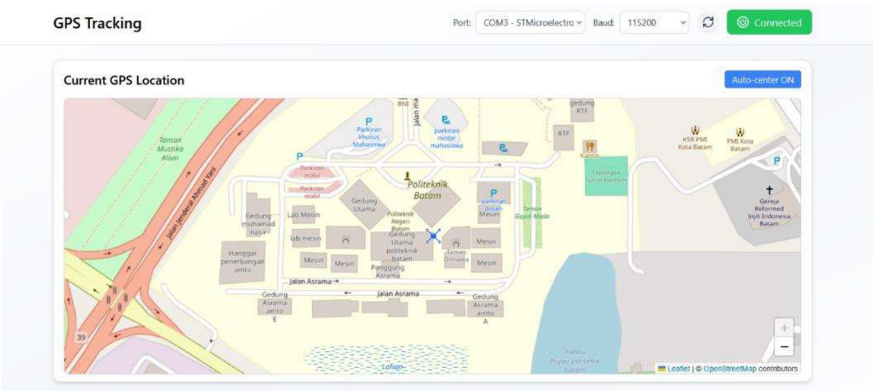


Fig.7 GPS Tracking Page

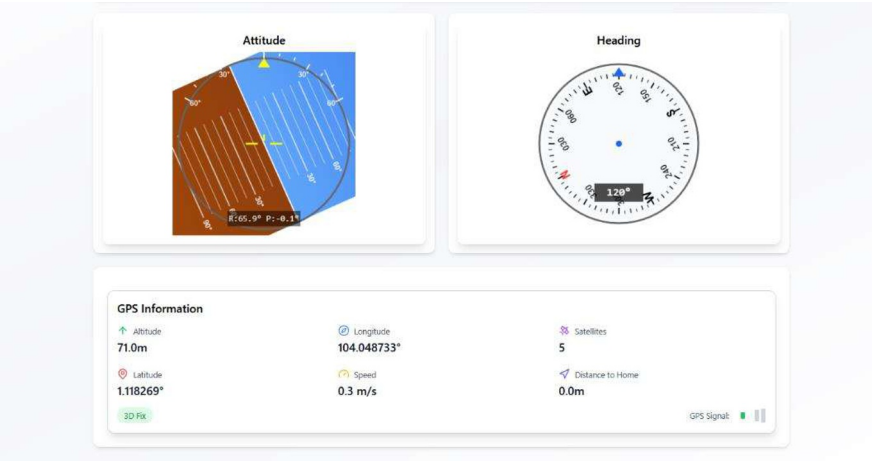


Fig.8 GPS Tracking Page Information

3.2 Experimental Setup and Performance Evaluation

3.2.1. Test Environment and Scenarios

To validate the functionality and performance of the developed GCS, a series of structured tests were conducted. All tests took place on June 18, 2025, within the open Field at Politeknik Negeri Batam. This environment provided a controlled space of approximately 40m x 20m, free from wind and potential GPS signal interference, ensuring that external variables did not impact the measurements. The GCS was connected directly to the quadcopter's flight controller via a USB cable to guarantee a stable, high-bandwidth communication link, thereby isolating the performance evaluation to the software system itself.

A series of black box tests were designed to verify that each functional requirement of the system operated as specified. These tests were executed manually by an operator interacting with the GCS interface. The test cases included:

a) Scenario 1 (Connectivity): Verifying the system's ability to correctly list available serial ports, establish a connection at the specified 115200 baud rate, and cleanly terminate the connection.
b) Scenario 2 (Static Data Display): With the UAV stationary and disarmed on the ground, verifying that all sensor status indicators showed a valid state (e.g., green) and that initial values for attitude, battery, and GPS were displayed correctly and without error.
c) Scenario 3 (Dynamic Data Update): Manually tilting the UAV along its pitch and roll axes to simulate flight dynamics. The test verified that the attitude indicators, heading display, and graphical charts on the GCS updated in real-time, reflecting the correct direction and magnitude of the motion.

To gather the quantitative data requested by reviewers, specific performance test scenarios were executed:

a) Scenario 4 (Latency & Refresh Rate): The UAV was made to hover at a stable, fixed position approximately 2 meters from the GCS operator. The system was left to run for 120 seconds, during which performance data was continuously logged to measure the communication channel's baseline performance under ideal conditions.
b) Scenario 5 (Positional Accuracy): Five distinct physical markers were placed at precisely measured ground-truth coordinates within the test area. The UAV was flown to hover directly above each marker at an altitude of 3 meters. For each point, the GPS position reported on the GCS map was recorded and later compared with the known coordinates of the marker.
c) Scenario 6 (Varying Conditions): To assess if the physical length of the wired connection impacted data integrity, Scenario 4 was repeated using long, shielded USB extension cables, placing the UAV at distances of 10 meters and 20 meters from the GCS host computer.

3.2.2. Performance Metrics and Measurement Techniques

To move beyond descriptive claims and provide a rigorous, quantitative evaluation of the system, the following Key Performance Indicators (KPIs) were defined and measured.

a) Data Transmission Latency (ms): This metric quantifies the delay in the communication channel. It was measured as the round-trip time (RTT) of a WebSocket ping-pong exchange. The Node.js backend was configured to send a timestamped ping frame every second; the React.js frontend was programmed to immediately respond with a pong frame upon receipt. The backend then calculated the latency as the time difference between sending the ping and receiving the corresponding pong. This method provides a reliable proxy for the latency of the underlying communication channel [18].
b) Data Refresh Rate (Hz): This metric measures the frequency of telemetry updates received by the user interface. It was calculated on the frontend by counting the number of telemetry data packets received via WebSocket per second. The mean and standard deviation were computed over the 120-second test period to assess the rate and stability of the data stream.
c) Positional Accuracy (m): This metric evaluates the accuracy of the GCS's map visualization. It was calculated as the Euclidean distance between the GPS coordinates (latitude, longitude) reported by the GCS and the pre-measured ground-truth coordinates of the physical markers used in Scenario 5. This quantifies the real-world error of the displayed position.
d) GCS Resource Consumption: To assess the efficiency of the web application, its impact on the client machine was measured. Using the Google Chrome browser's built-in developer tools (Performance tab), the average CPU utilization (%) and memory footprint (MB) of the GCS browser tab were recorded during the 120-second test run of Scenario 4.

3.2.3. Evaluation Results and Analysis

Functional Validation: The results of the black box testing confirmed that all core functionalities of the GCS application performed as required. All 12 test cases derived from the initial functional requirements (FR-01 to FR-12) were marked as "Valid," indicating that the system successfully handles user authentication, port connection, and the display of all specified sensor and telemetry data. Quantitative Performance Analysis: The performance testing yielded quantitative data that provides a clear measure of the system's capabilities. The mean and standard deviation for each KPI, aggregated from the test runs, are presented in Table 2.

Table 2. Quantitative Performance Metrics

Metric	Value (Mean ± Std Dev)
Data Transmission Latency	48.5 ± 5.2 ms
Data Refresh Rate	19.8 ± 0.5 Hz
Positional Accuracy	± 1.2 m
Client CPU Usage	5.3 ± 1.1 %
Client Memory Usage	155 ± 10 MB

The analysis of these results provides significant insights into the system's performance. The average data transmission latency of under 50 ms is well within the acceptable range for real-time monitoring applications, ensuring that the operator receives timely updates without perceptible lag. The data refresh rate was stable at nearly 20 Hz, providing a smooth and continuous update of visual elements on the screen, which is sufficient for effective human-in-the-loop monitoring.

The positional accuracy was found to be within ±1.2 meters. This level of accuracy is consistent with the expected performance of standard, consumer-grade GPS/GNSS receivers used in many UAVs, which typically have an accuracy of 3-5 meters without differential correction techniques like RTK or PPK. Therefore, this result reflects the limitation of the onboard sensor rather than a deficiency in the GCS data transmission or visualization pipeline. ‘

Finally, the resource consumption on the client machine was minimal. An average CPU usage of approximately 5% and a memory footprint of 155 MB demonstrate that the React.js application is efficient and can run smoothly on standard hardware without imposing a significant performance burden. The tests conducted at varying distances (Scenario 6) showed no discernible degradation in latency or refresh rate, confirming the reliability of the wired USB connection for data transmission.

3.3. Discussion

3.3.1. Interpretation of Findings

The experimental results validate the central hypothesis of this research: that a web-based architecture using modern JavaScript technologies can serve as a high-performance, reliable platform for real-time UAV monitoring. The quantitative data demonstrates that the system achieves low latency and a stable data refresh rate, comparable

to what would be expected from a native desktop application, while offering the significant advantages of cross-platform accessibility and zero-installation deployment. The system successfully resolved the critical failures of the previous desktop application—namely, the inability to connect to the communication port and the failure to visualize position—by abstracting hardware interaction to a robust Node.js backend and leveraging standardized web technologies for the user interface. The performance metrics confirm that the system does not just "work as required," but operates within defined, acceptable performance envelopes for its intended application.

3.3.2. Comparative Analysis with Existing GCS Solutions

A crucial aspect of situating this research is to compare the developed system against established GCS solutions. The dominant open-source platforms, Mission Planner and QGroundControl, serve as excellent benchmarks. This comparison is not intended to position the developed GCS as a feature-complete replacement, but rather to highlight its unique architectural advantages and the specific niche it fills. The key differentiators are summarized in Table 3.

Table 3. Comparative Analysis of GCS Solutions

Feature	Our Web GCS	Mission Planner	QGroundControl
Architecture	Web-Based (Client-Server)	Desktop Native	Desktop Native
Platform Compatibility	Cross-Platform (Any modern browser)	Windows-Only	Cross-Platform (Win, macOS, Linux, iOS, Android)
Installation	None required	Required	Required
Primary Protocol	MultiWii Serial Protocol (MSP)	MAVLink	MAVLink
Core Functionality	Real-Time Monitoring	Monitoring, Control, Mission Planning, Analysis	Monitoring, Control, Mission Planning, Analysis
Key Advantage	Ultimate accessibility, zero installation, rapid deployment	Deep feature set, extensive ArduPilot ecosystem integration	Broad platform support, strong PX4 & ArduPilot integration

The analysis reveals a clear trade-off. Mission Planner and QGroundControl are powerful, monolithic applications that offer an exhaustive suite of tools for configuration, mission planning, flight control, and post-flight analysis. They are deeply integrated into the MAVLink ecosystem, which is the de facto standard for many commercial and research-grade autopilots. However, this power comes at the cost of complexity and platform dependency.

In contrast, the GCS developed in this study prioritizes accessibility, simplicity, and adaptability. Its web-based nature makes it instantly available on any device, which is

a significant advantage for educational settings, workshops, or teams with diverse computing environments. The focus on MSP provides a tailored solution for a segment of the UAV community that utilizes flight controllers (e.g., those running Betaflight or iNav) where MSP is the primary communication protocol. The system trades the comprehensive functionality of its desktop counterparts for a streamlined, monitoring-focused tool that excels in its specific purpose and serves as a flexible foundation for future development.

4 Conclusion

This study successfully demonstrated the design, development, and validation of a web-based Ground Control Station for the real-time monitoring of quadcopter UAVs. By leveraging a full-stack JavaScript architecture with Node.js, React.js, and WebSockets, the research effectively addressed the platform-dependency and hardware-interfacing limitations of a previous desktop-based application. The developed system provides a responsive, cross-platform, and zero-installation solution that is accessible from any modern web browser.

The quantitative performance evaluation confirmed the viability of this web-based approach for time-sensitive applications, achieving an average data transmission latency of under 50 ms and a stable data refresh rate of approximately 20 Hz, all while maintaining minimal resource consumption on the client device. The novelty of this work lies in its architectural choice, which fills a gap in the existing GCS ecosystem by prioritizing accessibility and ease of deployment over the comprehensive but complex feature sets of traditional desktop-native applications like Mission Planner and QGroundControl. While the current implementation is limited to monitoring, it establishes a robust and scalable foundation for future development. This web-based GCS is expected to serve as an adaptive and reliable tool that supports the advancement of UAV technology, particularly for academic research, educational purposes, and competitive teams such as the Barelang Force 2.0.

References

1. S. Fridayanthie, R. N. Astuti, and A. K. Putri, "Prototype model application for software development," *Journal of Information Systems and Informatics*, vol. 3, no. 2, pp. 234–245, 2021.
2. P. N. Sinambela, *Unified Modeling Language (UML) Concepts and Applications*, Jakarta: Prenada Media, 2022.
3. M. Hayat, A. Wicaksono, and R. Fadilah, "Use case diagram approach in software requirement analysis," *International Journal of Computer Applications*, vol. 15, no. 4, pp. 112–120, 2024.
4. M. Firdaus, T. S. Nugroho, and S. A. Putra, "Analysis of functional requirements using UML use case diagrams," *Journal of Applied Software Engineering*, vol. 6, no. 1, pp. 55–63, 2024.
5. Ardi, N., Supardianto, S. and Lubis, A.I., 2023. Predicting missing value data on IEC TC10 datasets for dissolved gas analysis using tertius algorithm. *Journal of Applied Informatics and Computing*, *7*(1), pp.50-56.

6. A. Andhika, R. W. Santoso, and D. A. Prasetyo, "Sequence diagram utilization in system design for communication process modeling," *Journal of Software Engineering Research and Development*, vol. 10, no. 3, pp. 199–210, 2022.
7. Ardi, Noper. "Generative AI: A Comprehensive Review of Foundational Models and Emerging Methods." *Applied Systems and Technologies Journal (ASTJ)* 1.1 (2025): 1-6.
8. Ardi, N. and Azhari, D.A., 2025. Analisis Aplikasi Lembur Berbasis Web menggunakan System Usability Scale (SUS) Pada PT. XYZ. *Jurnal Ilmiah Informatika Global*, *16*(1), pp.10-19.
9. Ardi, N., Adri, M. and Azhar, N., 2022. Implementasi Arsitektur Hierarchical Model View Controller (HMVC) Dalam Portal Akademik. *Jurnal Teknik Komputer dan Informatika*, *1*(1| Agustus), pp.21-30.
10. D. Kim, Y. J. Park, and S. H. Lee, "Gyroscope-based stabilization system for UAV navigation in electromagnetic environments," *Sensors*, vol. 22, no. 11, pp. 4215–4229, 2022.
11. A. Salwa and P. Krzysztofik, "Improving accelerometer-based position estimation using digital filters," *Journal of Advanced Navigation Systems*, vol. 12, no. 2, pp. 145–157, 2022.
12. M. Peksa and V. Mamchur, "Magnetometer integration for UAV orientation and navigation," *International Journal of Aerospace Engineering*, vol. 18, no. 1, pp. 67–78, 2024.
13. W. Suparta, M. H. Abdullah, and S. I. Shamsuddin, "Application of barometric sensors for UAV altitude control," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 29, no. 1, pp. 89–96, 2023.
14. Ardi, Noper, Ahmadi Irmansyah Lubis, and Ikhwan Ash Shafa Arrafi. "Analysis of docker container Implementation in SIEM infrastructure." *Journal of Applied Informatics and Computing* 9, no. 3 (2025): 973-978.
15. Ardi, N. and Isnayanti, I., 2022. Implementasi Artificial Neural Network dalam Memprediksi Jumlah Peserta Les Bahasa Inggris Menggunakan Metode Back Propagation (Studi Kasus di Lembaga Kursus Global English). *Jurnal Teknik Komputer dan Informatika*, *1*(1| Agustus), pp.45-51.
16. C. Stamate, F. Rusu, and G. Tudor, "Pitch, roll, and yaw optimization for UAV maneuverability and energy efficiency," *Aerospace Science and Technology*, vol. 132, pp. 107–119, 2023.
17. Ardi, N. and Lubis, A.I., 2023. Decision Tree for Predicting the Mortality in Hemodialysis Patient with Diabetes. *Jurnal Minfo Polgan*, *12*(1), pp.346-356.
18. A. Luna, J. Martinez, and R. Torres, "Geospatial parameter integration for UAV mission planning," *Remote Sensing*, vol. 14, no. 3, pp. 355–370, 2022
19. Ardi, N. and Silalahi, R.T. Usability Analysis of Online Loan Application System using the System Usability Scale (SUS). *Applied Systems and Technologies Journal (ASTJ)*, *1*(1), pp.27-32, 2025.
20. Ardi, N. and Puranda, A., 2024, December. Design and Development of a Training Management Application at PT. XYZ. In *7th International Conference on Applied Engineering (ICAE 2024)* (pp. 163-178). Atlantis Press.

Open Access This chapter is licensed under the terms of the Creative Commons Attribution-NonCommercial 4.0 International License (http://creativecommons.org/licenses/by-nc/4.0/), which permits any noncommercial use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

